- The filter pipeline provides hooks into an MVC request so you can run functions at various points within an MVC request. With filters you can run code at specific points in the MVC process across all requests or a subset of requests. This is particularly useful for handling cross-cutting concerns that are specific to MVC.
- The filter pipeline executes as part of the MVC or Razor Pages execution. It consists of authorization filters, resource filters, action filters, page filters, exception filters, and result filters. Each filter type is grouped in a stage and can be used to achieve effects specific to that stage.
- Resource, action, and result filters run twice in the pipeline: an `*Executing` method on the way in and an `*Executed` method on the way out. Page filters run three times: after page handler selection, and before and after page handler execution.
- Authorization and exception filters run only once as part of the pipeline; they don't run after a response has been generated.
- Each type of filter has both a sync and an async version. For example, resource filters can implement either the `IResourceFilter` interface or the `IAsync-ResourceFilter` interface. You should use the synchronous interface unless your filter needs to use asynchronous method calls.
- You can add filters globally, at the controller level, at the Razor Page level, or at the action level. This is called the *scope* of the filter. Which scope you should choose depends on how broadly you want to apply the filter.